

From Docker to Kubernetes & CI/CD

Kubernetes • Minikube • kubectl • Git/GitHub • Jenkins • Docker • Docker Hub

1. Start from What Students Already Know

Students already know Dockerfile → Image → Container.

They can tag and push images to Docker Hub.

Docker solves packaging and container execution.

Now move from managing individual containers to managing an application at scale.

KEY IDEA

Docker = containerization

Next problem = management of many containers

This is where orchestration begins.

2. Why Do We Need Kubernetes?

Manual container management becomes difficult as systems grow.

A failed application instance may need replacement.

More users may require more application instances.

Users need a stable way to reach available application instances.

We need a system that coordinates these tasks.

ASK STUDENTS

Who restarts failed containers?

Who keeps the required number running?

How do users reach changing instances?

3. Container Orchestration

Container orchestration means automated coordination and management of containerized applications.

Typical concerns: deployment, scaling, recovery, networking and service access.

Kubernetes is a container orchestration platform.

Containerization packages/runs the app; orchestration manages how containerized apps operate as a system.

EXAM FOCUS

Define container orchestration.

Containerization $\neq$ orchestration

Example platform: Kubernetes

4. What Is Kubernetes?

Kubernetes (K8s) is an open-source platform for deploying and managing containerized applications.

You specify a desired state, such as how many application replicas should exist.

Kubernetes continually works to keep the actual state close to that desired state.

It supports scheduling, scaling, service discovery and self-healing.

REMEMBER

Kubernetes does not build your source code.

It consumes container images.

It manages workloads in a cluster.

5. Docker vs Kubernetes

Docker helps build images and run containers.

Kubernetes orchestrates containerized workloads across a cluster.

Docker image = packaged application.

Kubernetes objects = instructions/state describing how the application should run.

They are complementary in our workflow.

EXAM COMPARISON

Docker → containerization

Kubernetes → orchestration

Do not say Kubernetes replaces Docker.

6. Kubernetes Cluster

A Kubernetes cluster is the environment in which Kubernetes manages workloads.

Conceptually it has a Control Plane and one or more Worker Nodes.

Control Plane manages the cluster.

Worker Nodes provide resources where application workloads run.

MENTAL MODEL

Control Plane = management side

Worker Node = workload side

Pods run on Worker Nodes

7. Control Plane

Concept-first lecture • Slide 8

Receives requests through the Kubernetes API.

Maintains information about cluster state.

Makes scheduling and cluster-level management decisions.

Controllers work toward the desired state.

For this short course, understand the role before memorizing internal component names.

WHAT DOES IT DO?

Accepts cluster requests

Makes management decisions

Coordinates desired state

8. Worker Node

Concept-first lecture • Slide 9

Worker Nodes are where application workloads run.

The Control Plane schedules Pods onto available nodes.

Nodes provide compute resources and support container execution/networking.

A production cluster may have multiple Worker Nodes.

EXAM FOCUS

Where do Pods run? → Worker Nodes

Control Plane manages; Nodes execute workloads.

9. Pod — Where the Container Runs

Concept-first lecture • Slide 10

A Pod is the smallest deployable unit in Kubernetes.

A Pod contains one or more closely related containers.

Containers in the same Pod share a network context and can share storage.

In beginner examples, one Pod commonly contains one application container.

Pods are normally managed by higher-level controllers such as Deployments.

CONTAINER vs POD

Container = runtime instance of an image

Pod = Kubernetes workload wrapper/unit

One Pod can contain multiple containers

10. Why Do We Need a Deployment?

Concept-first lecture • Slide 11

Creating a single Pod directly does not provide the management behavior we usually want for an application.

A Deployment describes the desired application workload using Pods.

It can maintain a chosen number of replicas.

It supports replacement of failed managed Pods and controlled updates.

KEY SENTENCE

Deployment manages Pods.

Example: replicas = 3

Desired count is maintained.

11. Self-Healing

Concept-first lecture • Slide 12

Suppose a Deployment requires 3 replicas.

If one managed Pod fails, only 2 remain.

Kubernetes detects the mismatch between desired and actual state.

A replacement Pod can be created to restore the desired replica count.

MENTAL MODEL

Desired = 3

Actual = 2

Kubernetes works back toward 3

12. Scaling

Concept-first lecture • Slide 13

Scaling changes how many application replicas are desired.

More replicas can provide more application instances when demand requires it.

Fewer replicas can reduce resource use when appropriate.

Scaling still depends on available resources and correct application design.

EXAM FOCUS

Scaling = change replica count

Horizontal scaling → more/fewer instances

Not automatically 'faster' in every case.

13. Why Do We Need a Service?

Concept-first lecture • Slide 14

Pods can be replaced, so individual Pod addresses are not a stable application endpoint.

A Service provides a stable network abstraction for a selected set of Pods.

Clients communicate with the Service instead of tracking individual Pods.

The Service can direct traffic to available matching Pods.

KEY SENTENCE

Deployment manages Pods.

Service provides stable access to Pods.

Service $\neq$ Deployment

14. Kubernetes Application Picture

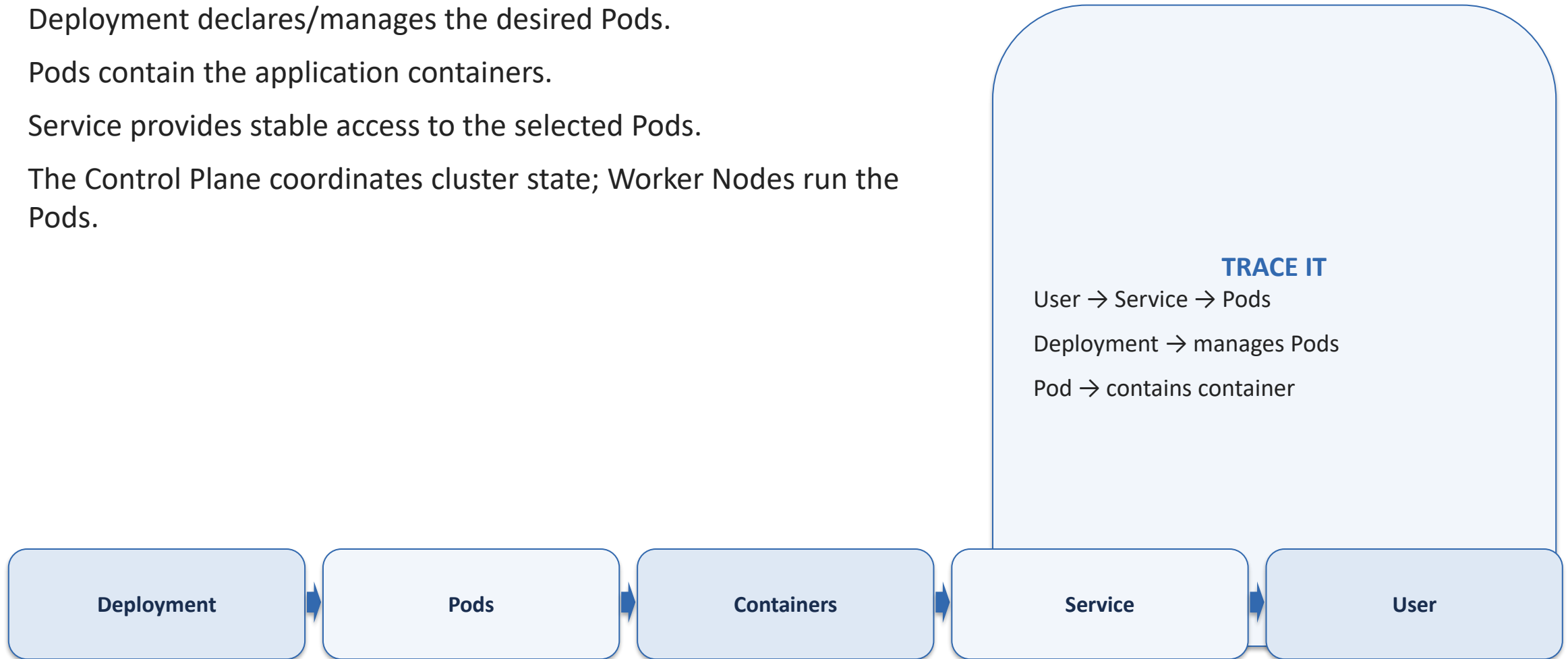
Concept-first lecture • Slide 15

Deployment declares/manages the desired Pods.

Pods contain the application containers.

Service provides stable access to the selected Pods.

The Control Plane coordinates cluster state; Worker Nodes run the Pods.



15. What Is kubectl?

Concept-first lecture • Slide 16

kubectl is the command-line client for interacting with a Kubernetes cluster.

It sends requests to the Kubernetes API.

It is used to create/apply, inspect, update and delete resources.

Practical examples will include get, describe, logs, apply and delete.

DO NOT CONFUSE

Kubernetes = platform

kubectl = CLI client

kubectl talks to the cluster

16. What Is Minikube?

Concept-first lecture • Slide 17

Minikube runs a local Kubernetes cluster for learning and development.

It is useful when students do not have a production multi-node cluster.

We use it to practice Kubernetes concepts on our local/VM environment.

Minikube is not Kubernetes itself; it is a tool for creating/running a local Kubernetes environment.

DO NOT CONFUSE

Minikube → local cluster tool

kubectl → cluster command client

Kubernetes → orchestration platform

17. Image to Running Application

Concept-first lecture • Slide 18

Source code is packaged into a Docker image.

The image can be stored in Docker Hub.

A Kubernetes Deployment references the image.

Kubernetes creates Pods that run containers from that image.

A Service can provide access to the application.

FOLLOW THE IMAGE

Code → Image

Image → Registry

Deployment → Pod → Container

Service → access

18. Imperative Commands vs Declarative YAML

Concept-first lecture • Slide 19

Imperative style tells Kubernetes an action through commands.

Declarative style describes the desired resource state in a YAML manifest.

YAML is common because configuration can be saved, reviewed and version-controlled.

kubectl apply can create/update resources from a manifest.

EXAM FOCUS

Imperative = tell action

Declarative = describe desired state

YAML supports repeatable configuration

19. Basic Kubernetes YAML Structure

Concept-first lecture • Slide 20

apiVersion: API version used by the resource.

kind: type of Kubernetes object, e.g., Deployment or Service.

metadata: identifying information such as name and labels.

spec: desired configuration/state of the resource.

Indentation matters in YAML.

READ, DON'T MEMORIZE

apiVersion

kind

metadata

spec

Understand structure first.

20. From Manual Work to CI/CD

Concept-first lecture • Slide 21

Without automation, a developer may manually build, test, create an image, push it and deploy it.

Repeating these steps manually can be slow and inconsistent.

A CI/CD pipeline automates selected stages of this software delivery workflow.

Automation should make the process repeatable and visible.

WHY AUTOMATE?

Repeatability

Faster feedback

Reduce manual repetition

Consistent workflow

21. Git and GitHub in the Workflow

Concept-first lecture • Slide 22

Git tracks source-code versions and changes.

GitHub hosts a shared remote Git repository.

Developers push code changes to the repository.

The repository can become the source from which an automated pipeline checks out code.

ROLE

Git = version control

GitHub = remote repository hosting

Source code lives here

22. What Is Continuous Integration (CI)?

Concept-first lecture • Slide 23

Developers integrate changes into a shared repository frequently.

Automation can check out, build and test those changes.

The goal is early feedback and repeatable integration.

A CI pipeline may also build a container image after successful checks.

EXAM FOCUS

CI → integrate/build/test frequently

Early feedback

Automation after code changes

23. What Is CD?

Concept-first lecture • Slide 24

Continuous Delivery automates the path to a deployable release; production release can still require approval.

Continuous Deployment automatically releases qualifying changes to production without a manual release approval step.

Both depend on reliable automation and testing.

In class, use 'CI/CD' as the overall pipeline term but know the distinction.

EXAM FOCUS

CI $\neq$ CD

Delivery may retain approval

Deployment automates production release

24. What Is Jenkins?

Concept-first lecture • Slide 25

Jenkins is an automation server used to implement CI/CD workflows.

It can execute pipeline stages and call tools/commands needed by the workflow.

Typical stages: checkout, build, test, Docker image build/push and deployment.

Jenkins coordinates automation; it does not replace Git, Docker or Kubernetes.

KEY ROLE

Jenkins = automation coordinator

Runs pipeline stages

Calls other tools

25. What Is a Jenkins Pipeline / Jenkinsfile?

Concept-first lecture • Slide 26

A Pipeline defines the sequence of automated stages.

A Jenkinsfile stores Pipeline definition as code in the project repository.

Example stages: Checkout → Build/Test → Docker Build → Push → Deploy.

Pipeline-as-code makes the automation workflow reviewable and repeatable.

MENTAL MODEL

Jenkins = automation server

Pipeline = workflow

Jenkinsfile = pipeline definition as code

26. Complete CI/CD Workflow

Concept-first lecture • Slide 27

Developer changes application source code.

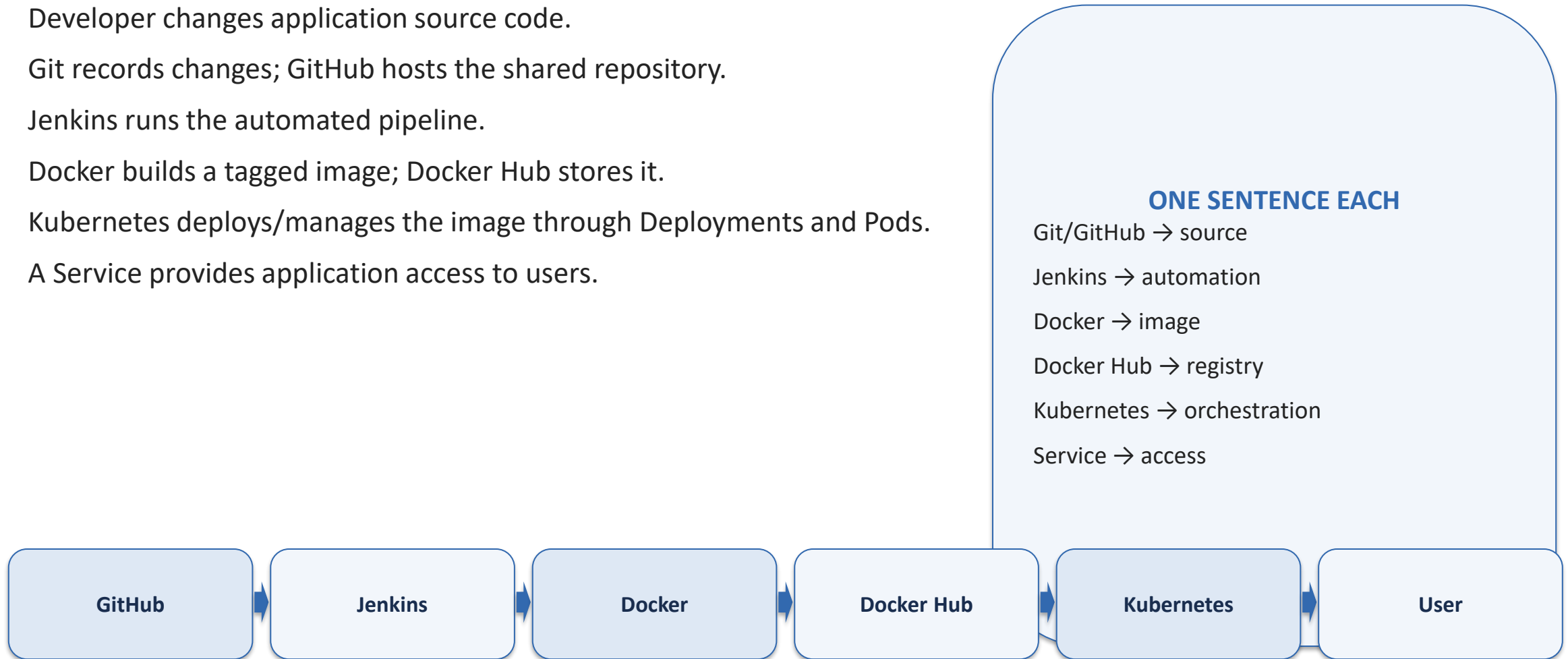
Git records changes; GitHub hosts the shared repository.

Jenkins runs the automated pipeline.

Docker builds a tagged image; Docker Hub stores it.

Kubernetes deploys/manages the image through Deployments and Pods.

A Service provides application access to users.



27. What Each Tool Does NOT Do

Concept-first lecture • Slide 28

GitHub is not a container image registry in this workflow.

Docker Hub does not manage source-code versions like Git.

Jenkins does not replace Docker or Kubernetes; it invokes/coordinates workflow steps.

Kubernetes does not normally build your application source into a Docker image.

Minikube is not the command-line client; kubectl is.

COMMON CONFUSIONS

GitHub ≠ Docker Hub

Jenkins ≠ Docker

Minikube ≠ kubectl

Docker ≠ Kubernetes

28. Comparisons

Concept-first lecture • Slide 29

Docker vs Kubernetes → containerization vs orchestration.

Container vs Pod → container runtime instance vs Kubernetes deployable workload unit.

Deployment vs Service → manage desired Pods vs stable network access.

Kubernetes vs Minikube → orchestration platform vs local cluster tool.

Kubernetes vs kubectl → platform vs CLI client.

GitHub vs Docker Hub → source repository hosting vs image registry.

CI vs CD → integrate/build/test vs delivery/deployment automation.

29. Practical Mental Map

Concept-first lecture • Slide 30

Start a local cluster → Minikube.

Communicate with cluster → kubectl.

Inspect Node/Pod/Deployment/Service → kubectl get/describe.

Create desired resources → commands or YAML + kubectl apply.

Troubleshoot → logs and describe.

Later connect GitHub → Jenkins → Docker/Docker Hub → Kubernetes.

BEFORE COMMANDS

Know the object first.

Then learn the command.

Do not memorize commands without purpose.

30. Final Picture to Remember

Concept-first lecture • Slide 31

Docker packages the application.

Kubernetes manages containerized applications.

Minikube gives us a local Kubernetes cluster.

kubectl lets us communicate with that cluster.

Git/GitHub manage and share source code.

Jenkins automates the delivery workflow.

Docker Hub stores images; Kubernetes runs/manages them for application access.

